Ling 285
Human Language and Technology
Lecture 9

Neural Networks 1

Khalil Iskarous

Department of Linguistics

University of Southern California

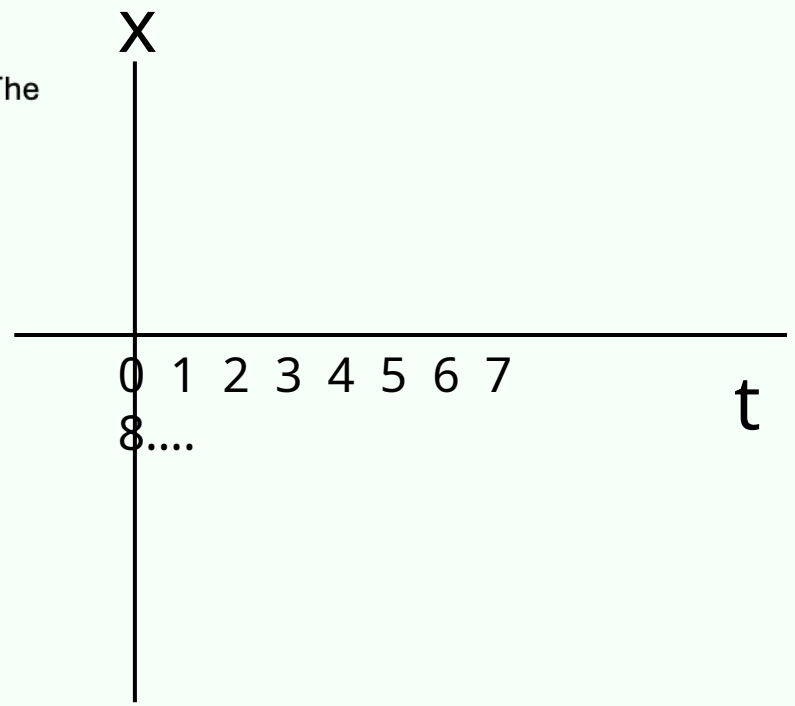
First Exam Next Thursday

Differential Equations are the mathematical expression of dynamical systems

- In 1676 Newton wrote a letter to Leibniz and expressed an idea in an anagram, a cryptographic object that would be revealed once he has fully proven the idea-- the way of estabilsihng priority in those days
- The secret: 6accdae13eff7i3l9n4o4qrr4s8t12ux
- Or in Latin: Data aequatione quotcunque fluentes quantitates involvente fluxiones invenire, et vice versa
- Or completely paraphrased: The laws of nature are expressed by differential equations (Vladimir Arnold paraphrasing).
- BTW: Inference and Learning in LLMs are differential equations.

Viscous spring, Negative/Balancing feedback

- $\dot{x} = -\frac{1}{3}x$
- $\dot{x}$ is *just* the change in x
- The point is to use this differential equation and an initial condition of x to figure out x for all time. The solution is a phenomenon in the world: SGD, flesh, temperature in a room…
- That is called the solution to the differential equation
- OK. Let's say x initial condition is at 100. So x at time t = 0 is 100.
- What does the equation say?
- The change in x is -1/3 of its current value x.
- What is -1/3 of x = 100, approximately?
- -33ish
- So the change in x is -33
- So if we start at 100, whand we lose 33, where do we get get to?
- 66ish!
- What calculation did we just do?
- $x_t = x_{t-1} - 1/3\ x_{t-1}$
- Now x is 66. So plug it into the equation again. What's 1/3 of 66? Take that away from 66
- Pop Quiz: Go to Brightspace, Enter quizzes, and write down the value of x at t = 2, 3, 4. You can use a calculator on the web or your computer
- That's Euler's Method! Differential Equations are solved to predict what the world is like using arithmetic!



First Exam Next Thursday

Now we will do this in Python!

Software 1.0





A Neural Network

- Imagine you are given many pictures of faces (48 Million pixels), and you're asked to identify one of 1000 possible people.
- Imagine you are given speech waves and you're asked to to identify for each, which of 25 sounds each speech wave corresponds to.
- What can do this? A Neural Network (NN)
- I will not tell you now why it's called a NN. I will teach you what it is by what it does– how it walks and how it swims.
- The job of an NN is to take a bunch of numbers the 48M pixels in one image/numbers of a wave and to return a probability distribution.
- For speech the probability distribution is 25 sounds
- But let's say it had just 5 vowels, this is a pdf: [.2,.2,.4,.2,0]
- And let's say we agree the vowel order is read, rad, rude, Rud, rod
- The p's must sum to 1 (100%)
- That says: there's 20% chance the sound is the v in read, 20% it's the vowel in rad, 40% the vowel in rude, 20% it's the vowel in Rud,

A Neural Network

- The input of an NN is a bunch of numbers, signal, or a *vector*, e.g., [4,1,-3,2,1,.4,1,2,-3,…] (usually 1000's of nums). Let's say these are the pressure numbers of a speech wave
- The output of the NN is a *decision* pdf: Given the signal, what's the p(each object): p of each vowels read, rad, rude, Rud, rod
- How? How can NNs do that? How can it *make a decision*.
- They have *knowledge*, in a bunch of weights w's
- And somehow the input signal vector is combined with these w's and a decision about the object detected is made, *eventually*, as there are many many w's and many many calculations! Several trillion w's and billions of calculations.
- These magical w's allow the NN to make decisions, but how does it know these w's???
- Fake it till you make it!
- SGD for NNs. You start out with random w's.
    1. Give it a signal (let's say we give it the vowel in *Rud)*
    2. Forward Propagation: It uses its stupid w's, and it produces a pdf that doesn't reflect reality: [.4,.1,.2,.2,.1]
    3. We tell it: NO!!!!!!! You should have decided: 0,0,0,1,0], so it realizes it made a big Error or Loss
    4. It improves its w's to reduce the Loss
    5. Give it another signal and repeat billions of times!

Templatic Perception and Hierarchical Perception

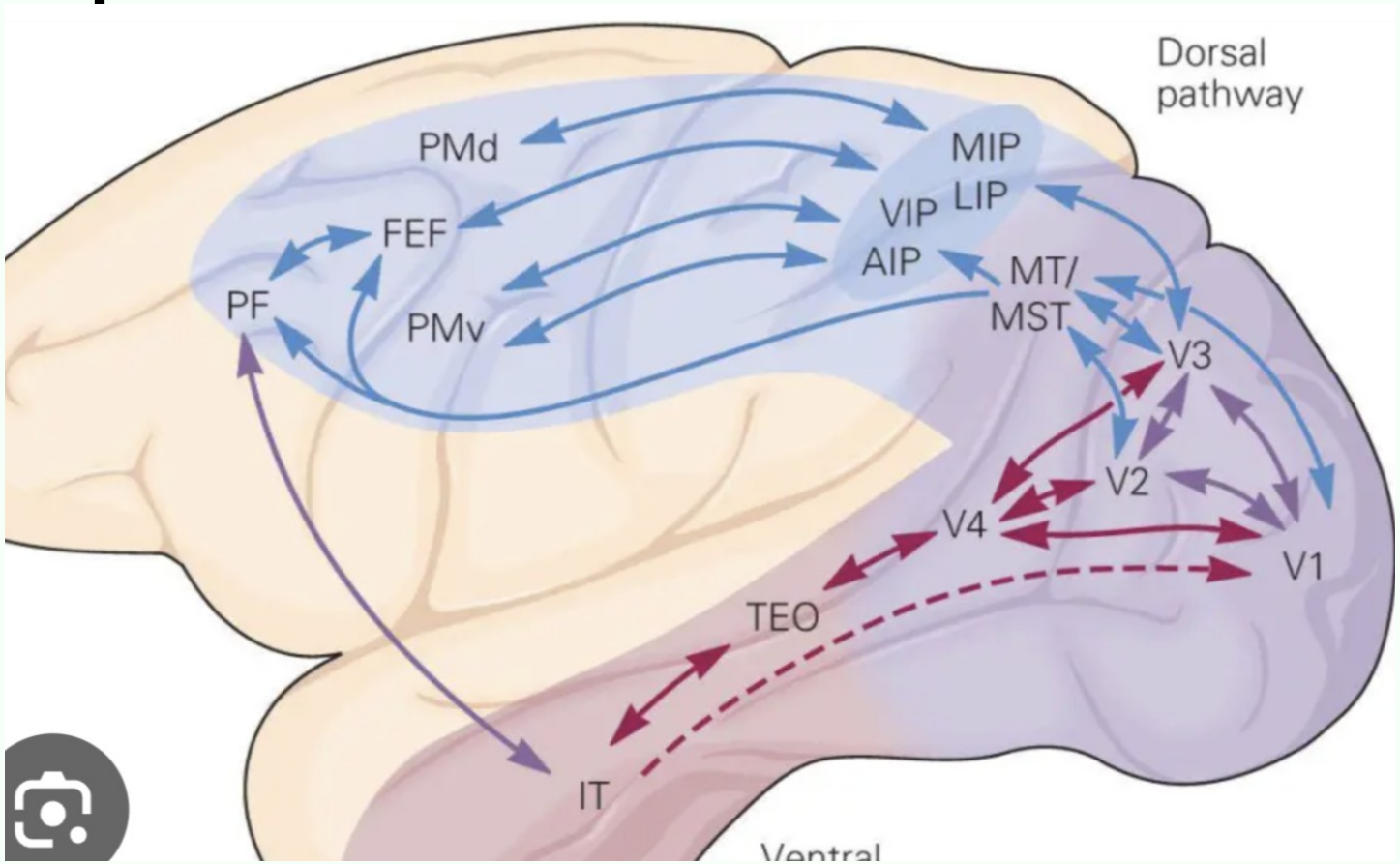
- Our Goal: Understand how an NN uses its weights to process the input to make the right decision at the output, e.g., when given a [z] speech signal (= waveform = pressure waveform), the correct decision should be [z], not [s], not [a], etc.
- Something useful to know is how humans perceive speech, faces, etc. And perhaps knowing how how our own biological neural networks perceive speech, we can use that to understand how NNs model this via an artificial computation.
- So there are two basic theories of animal perception that inform NN design:
  - Templatic Perception
  - Hierarchical Perception

Templatic Perception

- Here's a theory of how face perception works
  1. The retina in the eye receives light information from the face
  2. The brain has a template, a picture of sorts, stroed in it, of every face that the person knows
  3. The brain compares the input face with each of its stored templates for faces
  4. The template with the highest ***similarity*** to the input face is what is perceived
  5. If there's a name associated with the winning template, that name is the perceptual decision
- What this theory implies for AI: The knowledge weights of NNs are like perceptual templates.
- Remmber this from before: So the NN ***combines its currrent weights*** **with the input** face pixels and produces a **Computed Output** that is wrong (when we haven't learnt anything)!
- So the way that an NN's weights/knowledge/templates are combined with the input is that we determine similarity between weights and inputs.
- This is difficult to understand now, but just keep this in mind.

Hierarchical Perception

- For a long time, psychologists and neuroscientists have known that perception is done in stages, hierarchically
- So the input leads to one new representation, then that representation becomes another, then another, till the output decision is made!



NNs are Templatic Hierarchical Perceivers

- Let’s first talk about the templatic aspect
- The key to templatic perception is that some internal template of numbers is compared to some input set of numbers to find out how similar the template is to the input
- NNs assume: Input is a list or *vector* of numbers and internal template weights are also a list or vector of numbers
- So how can we compare two lists of numbers to determine if they are similar or not.
- E.g.
  - Input = [-1 3 -2]
  - Temp1 = [-2 0 3]
  - Temp2 = [-2   2 -1]
- Is list Input more similar to template 1 or 2?
- Inner product:
  - Multiply 2 lists component-wise
  - Add all products
  - If IP is near 0, the lists are dissimilar
  - If IP is large and positive, lists are similar

Neural Networks

- Inference: We give the system a signal vector, the w's are inner producted (template matching) over and over again (hierarchical perception) like Euler's algorithm ☺
- Learning: We give the system a signal a vector whose identity we know, we ask it to *infer* its identity, with initial random w's. It can't. So the w's are improved via SGD!